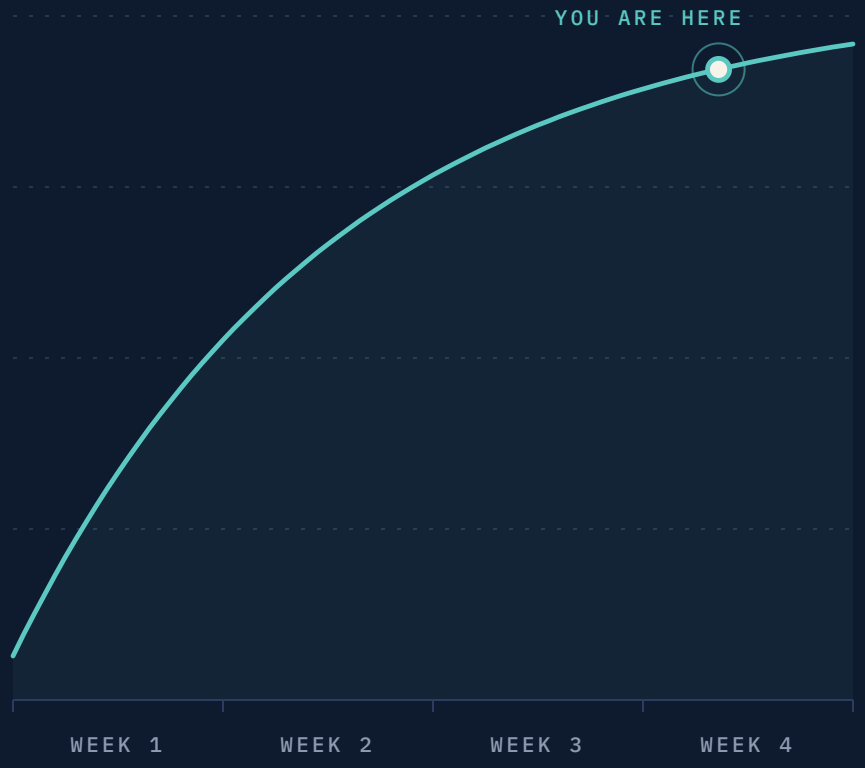
WEEK 4 · DAY 2, PART 1 · OVERVIEW

Analyzing Text, Part 1



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 6 SECTIONS / 7 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

We keep every FOMC statement and have only ever read them. Before we run all 46, prove the tools on one, 2023-11-01: what figures does it state, how often does it say "rate", the word, and who voted? I can count the names myself, so your count must agree with mine. Then our own book: the broker sends bond descriptions as one line of text, and I want a table; and every CUSIP in the holdings file must have the right shape. Counts only: no reading of policy.

ABOUT 50 MIN WITH EXERCISES / 6 SECTIONS / 7 EXERCISES

Your Route Through Day 2, Part 1

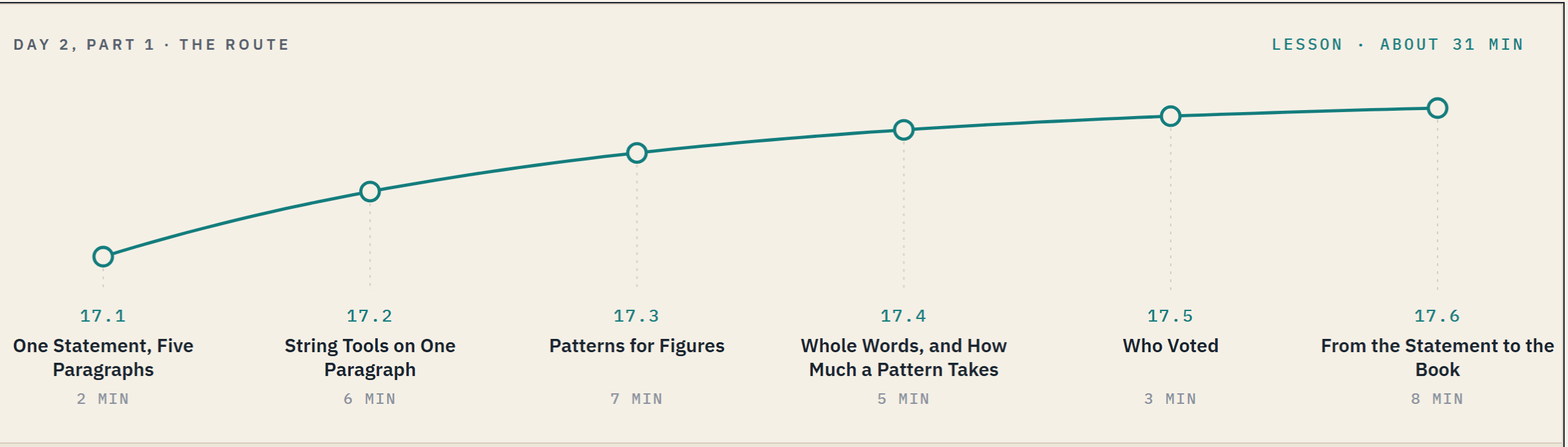
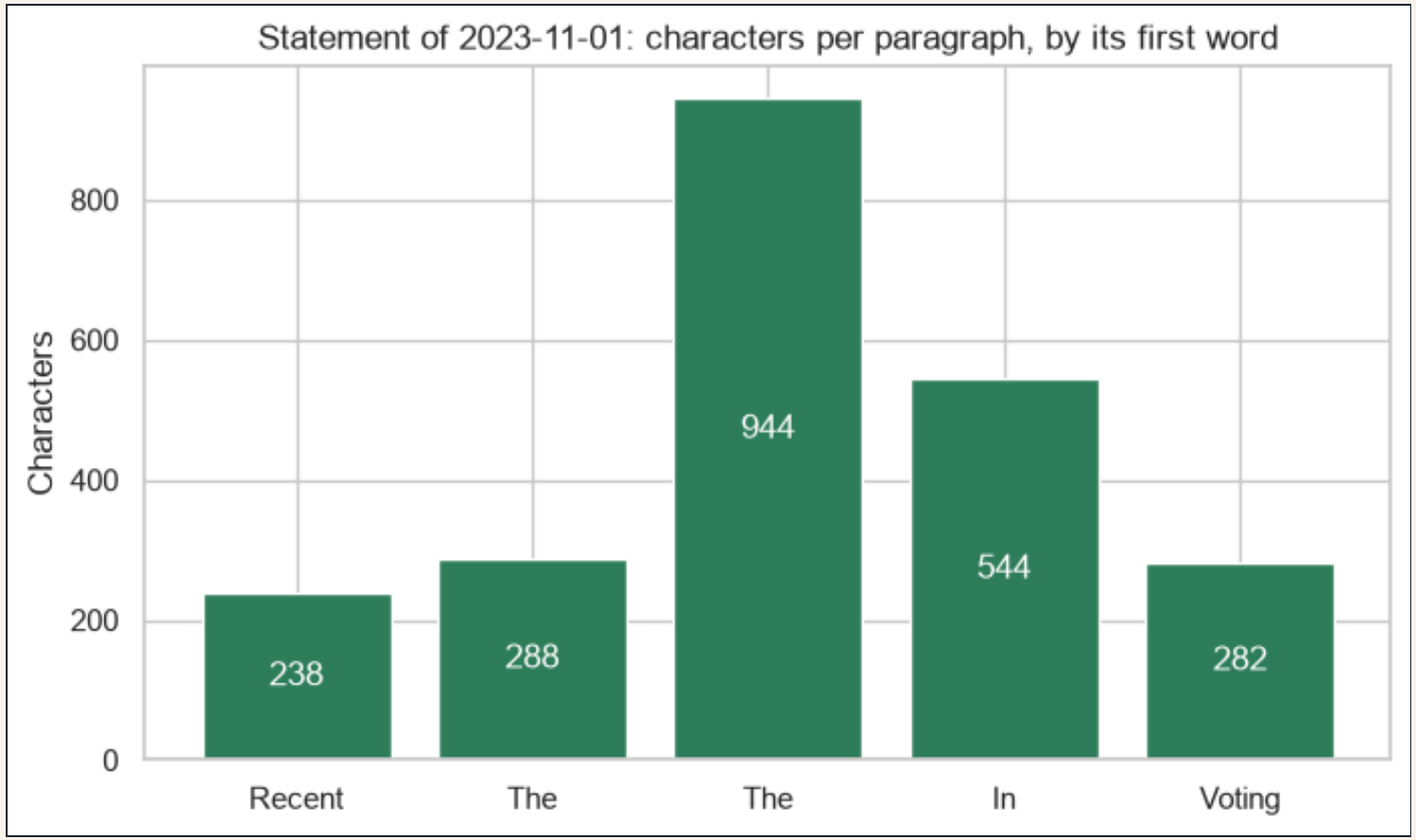


FIGURE 1 Six sections, in order, with about how long each takes.

Time. About 31 minutes for the lesson, then about 21 for the seven exercises.

17.1 One Statement, Five Paragraphs



CUT IT INTO PARAGRAPHS

```
paragraphs = statement.split('\n')
print(type(statement), len(paragraphs))
```

```
<class 'str'> 5
```

Figures in the third, voters in the last. Start with the first.

17.2 String Tools on One Paragraph

WHERE A WORD STARTS

```
position = opening.find('Inflation')
print(position)
print(opening[:6], opening[position:position + 9], opening[-9:])
```

```
211
Recent Inflation elevated.
```

WORDS

```
words = opening.split()
print(len(words), words[-3:])
```

```
37 ['Inflation', 'remains', 'elevated.']
```

JOIN THEM BACK

```
print(' '.join(words[:5]))
print('-'.join(words[:5]))
```

```
Recent indicators suggest that economic
Recent-indicators-suggest-that-economic
```

Every tool here looks for one exact piece of text. A figure changes its digits every meeting.

17.3 Patterns for Figures

```
range_pattern = r'[\d/-]+ to [\d/-]+ percent'
```

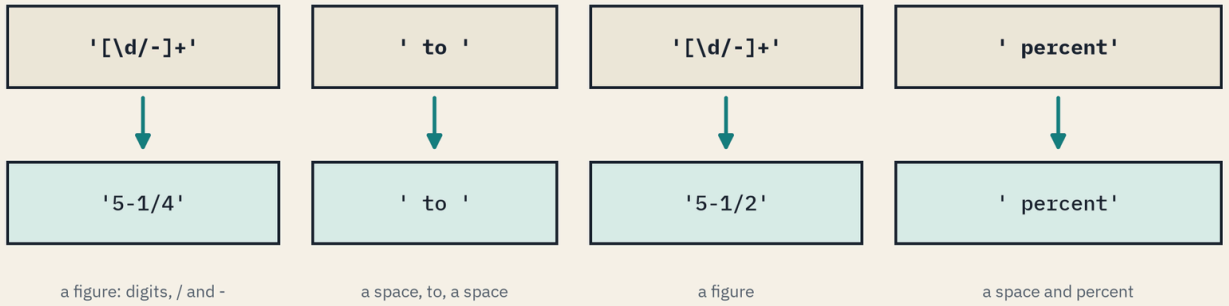


FIGURE 4 The range pattern over 5-1/4 to 5-1/2 percent, each piece bracketed to the characters it takes.

FIND THE RANGE

```
range_pattern = r'[\d/-]+ to [\d/-]+ percent'
print(re.findall(range_pattern, policy))
```

```
['5-1/4 to 5-1/2 percent']
```

WHOLE FIGURES

```
figures = re.findall(r'[\d/-]+ percent', policy)
```

```
['2 percent', '5-1/2 percent', '2 percent', '2 percent']
```

The figures: 2 percent three times, and a range of 5-1/4 to 5-1/2 percent.

17.4 Whole Words, and How Much a Pattern Takes

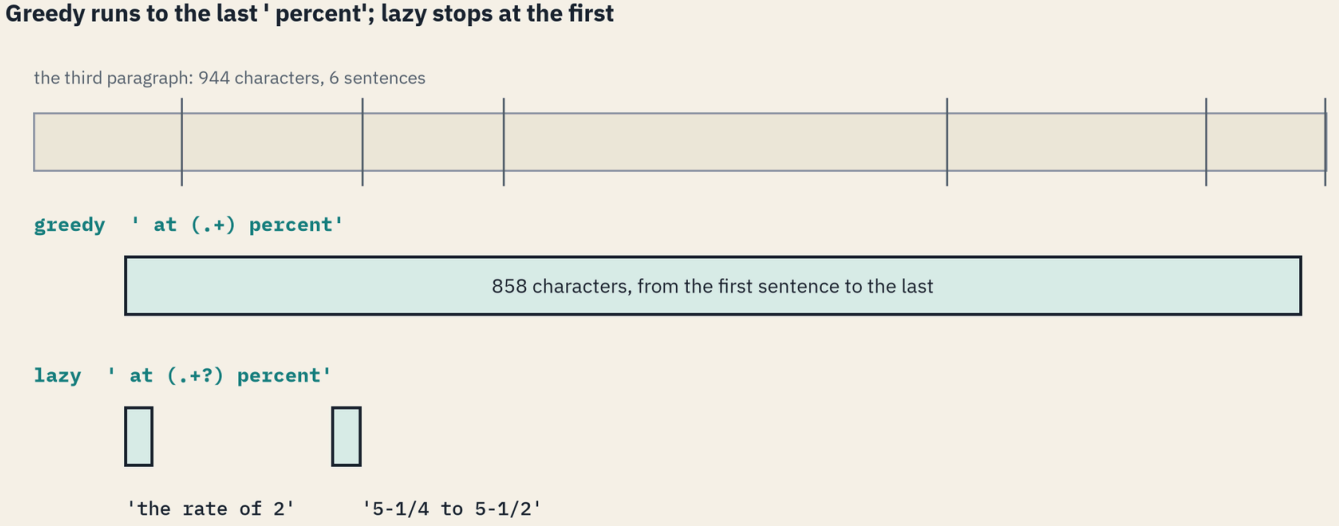


FIGURE 5 Of the paragraph's 944 characters, the greedy match takes 858; the two lazy matches take only the figures.

GREEDY, THEN LAZY

```
print(len(greedy), 'of', len(policy), 'characters')
print(lazy)
print(re.findall(r' at (.+?) percent', policy))
```

```
858 of 944 characters
the rate of 2
['the rate of 2', '5-1/4 to 5-1/2']
```

The range pattern cannot run on like this: a letter is not in its brackets.

When Nothing Matches

WHEN NOTHING MATCHES

```
re.search(r'\d+ basis points', policy).group(0)
```

```
AttributeError: 'NoneType' object has no attribute 'group'
```

HANDLE IT

```
match = re.search(r'\d+ basis points', policy)
if match is None:
    print('No figure in basis points in this paragraph.')
else:
    print(match.group(0))
```

```
No figure in basis points in this paragraph.
```

EXCEL SIDE BY SIDE

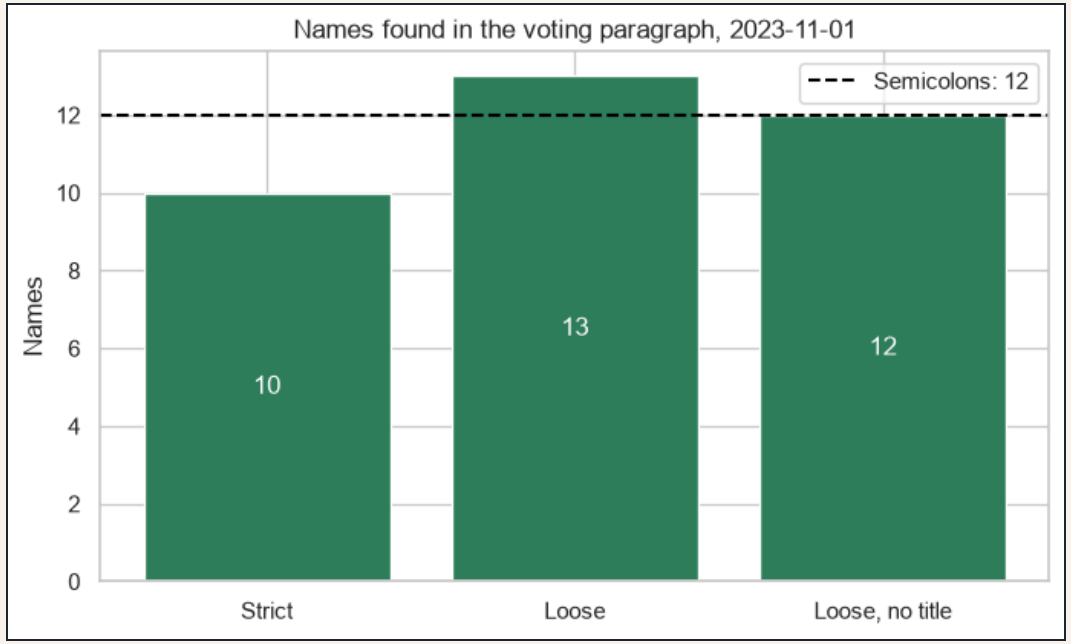
JOB	PYTHON	EXCEL
The first match	`re.search(pattern, text).group(0)`	`=REGEXEXTRACT(A2, pattern)`: `5-1/4 to 5-1/2 percent`
Between two markers	`re.search(r' at (.+?) percent', text).group(1)`	`=TEXTBEFORE(TEXTAFTER(A2, " at "), " percent")`: `the rate of 2`, the lazy answer
Cells holding a piece	`count()` on each word	`=COUNTIF(B2:B38, "*rate*")` on the 37 words: 2

17.5 Who Voted

COUNT THE NAMES

```
voting = paragraphs[-1]
print('By semicolons:', voting.count(';') + 1)
strict = re.findall(r'[A-Z][a-z]+ [A-Z]\. [A-Z][a-z]+', voting)
print('By the pattern:', len(strict))
```

```
By semicolons: 12
By the pattern: 10
```



The count from the semicolons decided which way each pattern was wrong.

17.6 From the Statement to the Book

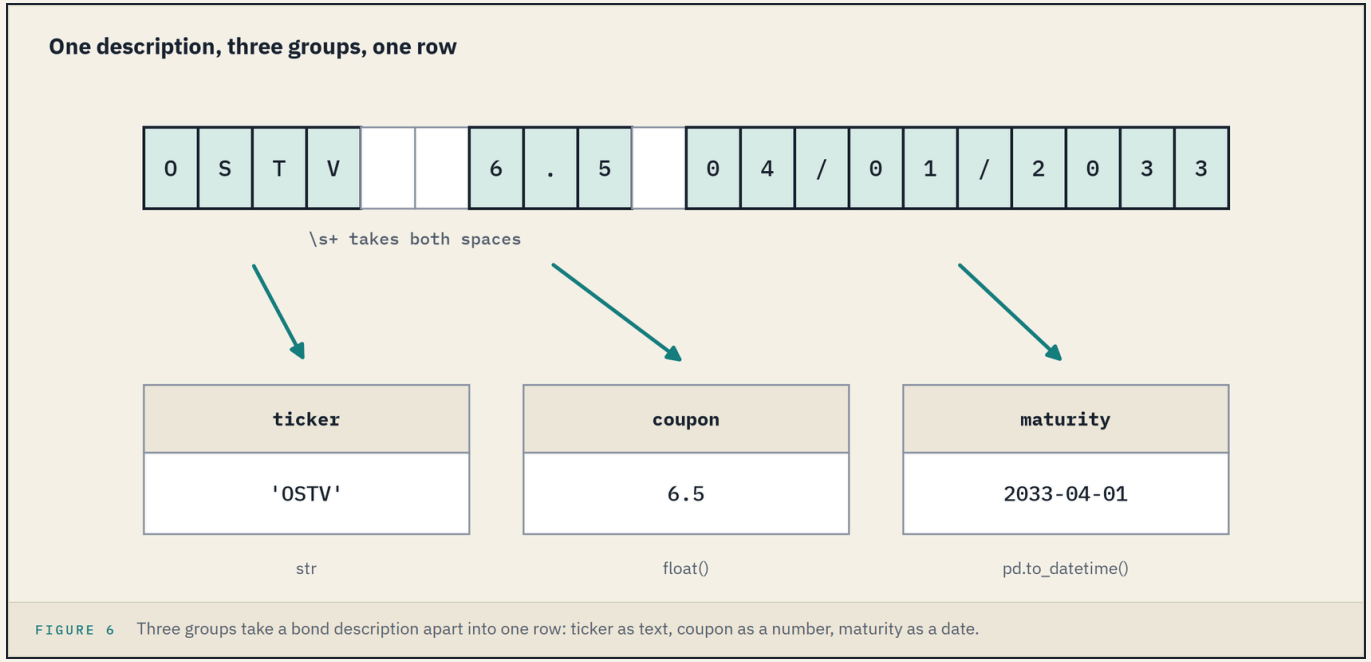


FIGURE 6 Three groups take a bond description apart into one row: ticker as text, coupon as a number, maturity as a date.

THE TABLE

```
print(bonds)

ticker  coupon   maturity
0   KRIX   7.875 2032-08-26
1   BEZE   7.000 2032-06-18
2   BEZH   8.125 2029-04-17
3   BEZL   5.125 2030-06-03
4   LORE   7.250 2030-03-08
5   OSTV   6.500 2033-04-01
6   SYLX   3.625 2055-09-03
7   VYRN   7.375 2037-12-12
6.609 2055-09-03
```

Eight lines of text became a table.

Every CUSIP in the Book

TEST A CUSIP AND FOUR DAMAGED COPIES

```
for value in [cusips.iloc[0], '99080cae8', '99080CAE', '99080CAE8 ', '99080-CAE8']:
    print(repr(value), re.fullmatch(cusip_shape, value) is not None)
```

```
'99080CAE8' True
'99080cae8' False
'99080CAE' False
'99080CAE8 ' False
'99080-CAE8' False
```

ALL 200 IN THE HOLDINGS FILE

```
cusip_shape = r'[0-9A-Z]{9}'
has_shape = cusips.str.fullmatch(cusip_shape)
print(len(cusips), has_shape.sum())
```

```
200 200
```

THE WRONG REPAIR

```
print(re.sub(r'[^0-9A-Z]', '', '99080cae8'))
```

```
990808
```

200 of 200 have the shape; the ninth character, a check digit, is a test for another day.

You Can Now

- Measure, search, count, split, join, and strip text.
- Build a regex from `\d`, `\w`, `\s`, `\b`, classes, `+`, `*`, `?`, anchors, and groups.
- Use `re.findall()`, `re.search()`, and `re.sub()`, and tie a pattern to an outside count.
- Tell greedy from lazy, and test a whole value with `re.fullmatch()`.
- Read bond descriptions into a table and test every CUSIP's shape.

NEXT
WEEK 4 · DAY 2, PART 2

Analyzing Text, Part 2

Clean a statement in steps, count its words, and do the same to a table of 46 statements.